WEEK 07

Exceptions and with-blocks

Python foundations

Exceptions and with-blocks

Exceptions, assignments and transaction boundaries

Exceptions interrupt

raise leaves the normal path; a matching except handles it.

```
try: raise ValueError("invalid")
except ValueError: print("handled")
```

Assignment waits

If the right side fails, the target keeps its previous value.

```
result = 7
# result = int("bad") would leave result at 7
```

Mutation is not rollback

Ordinary Python changes stay made even if later code fails.

```
items = []; items.append("A")
# A later exception does not undo append.
```

Stopping is not the same as undoing

1. Stock note

`count = 5`

5

stored count

2. Remove one

`count -= 1`

4

stored count

3. Later step fails

`ValueError`

4

count stays 4

```
state["count"] -= 1
raise ValueError("later step failed")
```

An exception changes control flow. Database rollback is a separate guarantee.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
raise ValueError(...)	Stop the current path	Signal a known invalid value
try / except	Choose an error path	Handle a specific exception
with db:	A transaction boundary	Observe normal or failed exit
assertRaises(...)	Check an expected error	Also inspect the resulting state

Python Basics · sections 09, 13, 17 · use only when you need a lookup.

Check exact type before a numeric range

```
value = True
print(type(value) is int)
value = "2"
print(type(value) is int and value > 0)
value = 2
print(type(value) is int and value > 0)
```

1. type(value) is int excludes text, None and Boolean True.

An exception does not undo ordinary mutation

```
state = {"count": 5}
try:
    state["count"] -= 1
    raise ValueError("later step failed")
except ValueError:
    print(state["count"])
assert state["count"] == 4
```

1. This deliberately changes a dictionary before raising.

A failed call leaves its assignment unfinished

```
def checked_count(value):
    if value < 0:
        raise ValueError("negative count")
    return value
count = 5
try:
    count = checked_count(-1)
except ValueError:
    print("rejected")
print(count)
```

1. Expected: rejected, then 5.
2. The right side raises before count is reassigned.
3. Change -1 to 2 and predict the new output.

assertRaises: check the exception and state

```
01  import unittest
02
03  def reserve_count(available, requested):
04      if requested > available:
05          raise ValueError("not enough containers")
06      return available - requested
07
08  check = unittest.TestCase()
09  available = 3
10  with check.assertRaises(ValueError):
11      available = reserve_count(available, 4)
12  check.assertEqual(available, 3)
```

1. assertRaises checks the exception type.
2. Failed assignment keeps the old value.
3. Database rollback needs its own state check.

Check the expected exception and the state left afterward.

Hands-on practice

Week_07.ipynb · Session 2

Python practice

P1–P2 · Core

Two small programs with approach hints.

P3 · Core

A complete program with normal and edge-case tests.

P4–P5 · Optional

An application and a harder challenge.

P1–P3: your code, expected results and an explanation. P4–P5 are optional.

Project studio

All-or-nothing reservation

The project need

1

Need

A booking must not leave only part of a request set aside.

2

Operation

Accept the whole booking, or leave the records unchanged if it cannot finish.

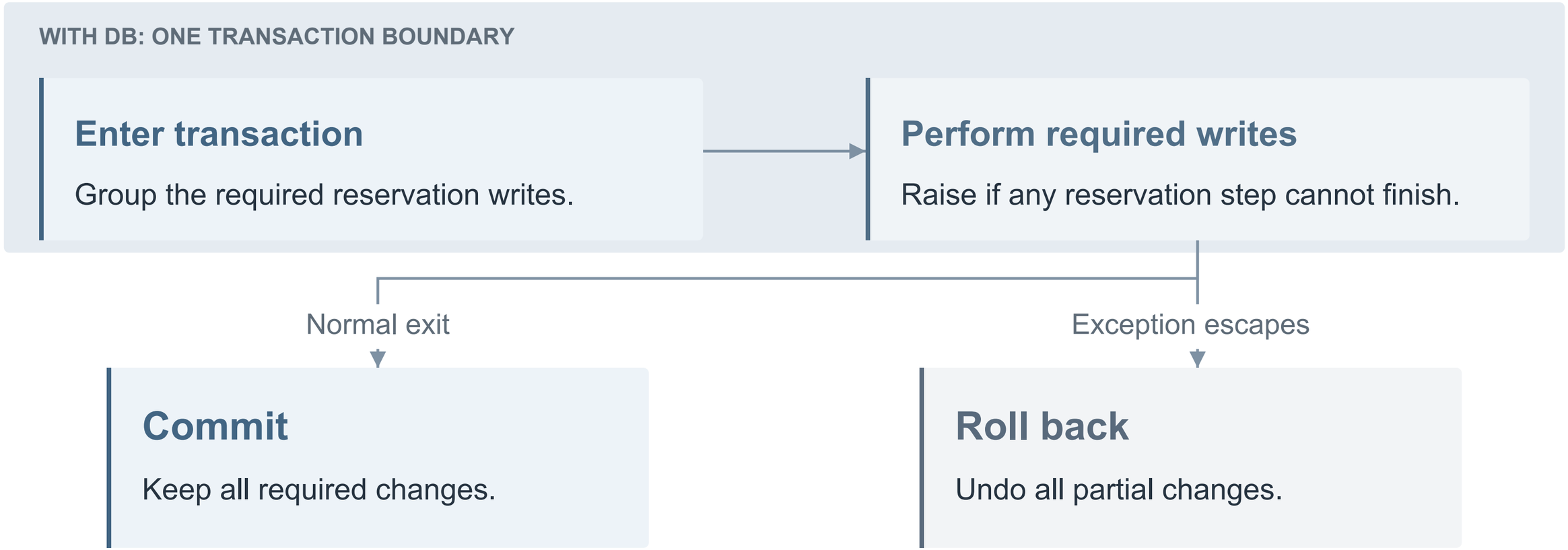
3

Result

Another volunteer sees consistent available counts.

One item unavailable → all counts stay unchanged.

Let a required failure escape before handling it



Catch the failure after the transaction has had a chance to roll back.

Reference: Observe rollback after a deliberate interruption

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE availability(containers INT)")
with db: db.execute("INSERT INTO availability VALUES (5)")
try:
    with db:
        db.execute("UPDATE availability SET containers=3")
        raise RuntimeError("practice interruption")
except RuntimeError: pass
row = db.execute("SELECT * FROM availability").fetchone()
assert row == (5,)
db.close()
```

1. Commit the opening state first.
2. Let failure escape the with block.
3. Read the state after handling the error.

An escaping exception rolls back writes in the transaction.

Conditional UPDATE with placeholders

```
UPDATE offer_batches
SET available_containers = available_containers - ?
WHERE batch_id = ?
  AND available_containers >= ?;
```

1. Bind requested count, batch ID, requested count
2. Inspect cursor.rowcount after execute
3. Zero does not automatically raise a failure

A zero-row update must be inspected and handled explicitly.

rowcount: one row changed

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE offers(id TEXT, available INT)")
db.execute("INSERT INTO offers VALUES ('DEMO',5)")
cur = db.execute(
    "UPDATE offers SET available=available-? "
    "WHERE id=? AND available>=?", (2, "DEMO", 2))
assert cur.rowcount == 1
row = db.execute("SELECT * FROM offers").fetchone()
assert row[-1] == 3
db.close()
```

1. Inspect the returned cursor.
2. rowcount is 1 after a match.
3. Query again to observe the value.

A matching conditional UPDATE changes one row.

rowcount: no row matched

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE offers(available INT)")
db.execute("INSERT INTO offers VALUES (3)")
cur = db.execute(
    "UPDATE offers SET available=available-? "
    "WHERE available>=?", (4, 4))
assert cur.rowcount == 0
row = db.execute("SELECT * FROM offers").fetchone()
assert row[-1] == 3
db.close()
```

1. rowcount is 0 when the condition fails.
2. The stored value stays the same.
3. The caller decides whether to raise.

No matching row means no update; it does not raise automatically.

Project task

Supplied starting point

Database fixture starts Q11=4, Q12=4; schema, starter and failure-injection hook are supplied.

You implement / do

Validate review, counts and batches; apply guarded updates in one transaction. Raise on shortage; let failure escape the with-block before handling it.

Check the evidence

Unreviewed, shortage and injected failure must leave state unchanged. Success Q11:2 + Q12:1 → remaining 2,3; one header and two lines.

Also check zero counts, unknown batches and repeated batch lines.

Project checks

Step	What to show
Implementation	Implement reserve in one transaction; reject bad requests.
Checks	Success leaves 2/3; refusals leave every table unchanged.

Also check zero counts, unknown batches and repeated batch lines.

Further reading in English

1. ## SQLite transactions

   https://www.sqlite.org/lang_transaction.html

2. ## Python sqlite3 connection context manager

   https://docs.python.org/3/library/sqlite3.html#how-to-use-the-connection-context-manager

Read one relevant section, then bring an example and a question.